**Cairo University** - Faculty of Engineering

EECE Department

**4th Year – Second Semester**

**Neural Networks ELC4028**

**Spring Semester of Academic Year: 2025 / 2026**

---

# Assignment 2

| Name | ID | Sec |
|---|---|---|
| ايريني حسني ماهر رمزي | 9220175 | 1 |
| محمد خالد محمد شتا | 9203230 | 3 |
| محب ميخائيل حشمت ميخائيل | 9220638 | 3 |

**Submitted to**

**Dr. Mohsen Rashwan**

**Dr. Mohammed Abdelghany**

**Date of Submission:** May 2026

# Contents


**1 Introduction** **3**

**2 Part I: Neural Networks on Image and Audio Data** **4**
2.1 Problem 1: Multi-layer Perceptron (MLP) and Feature Extraction on ReducedMNIST 4
2.1.1 Part A: MLP Hyperparameter Tuning (Raw Pixels) . . . . . 4
2.1.2 Part B: Classifier Comparison using Extracted Features . . . . . 4
2.1.3 Observations on Extracted Features . . . . . 5
2.2 Problem 2: Convolutional Neural Network (CNN) on ReducedMNIST . . . . . 6
2.2.1 CNN Architecture and Variations . . . . . 6
2.2.2 Comparison with Assignment 1 and Problem 1 . . . . . 6
2.2.3 Discussion . . . . . 6
2.3 Problem 3: Speech Recognition from Speech Spectrum . . . . . 7
2.3.1 Network and Input Representation . . . . . 7
2.3.2 Required Augmentation Experiments . . . . . 7
2.3.3 Discussion . . . . . 8

**3 Part II: Data Augmentation and Synthesis** **9**
3.1 Problem 4: Autoencoder for Utterance Representation . . . . . 9
3.1.1 Baseline Average-Frame Representation . . . . . 9
3.1.2 AutoEncoder Representation . . . . . 9
3.1.3 Classifier Results . . . . . 9
3.1.4 Discussion . . . . . 10
3.2 Problem 5: Data Augmentation on ReducedMNIST . . . . . 11
3.2.1 Augmentation pipeline . . . . . 11
3.2.2 LeNet-5 model (as used) . . . . . 11
3.2.3 Evaluation metrics . . . . . 11
3.2.4 Results . . . . . 12
3.2.5 Discussion . . . . . 12
3.2.6 Conclusions . . . . . 13
3.3 Problem 6: Generative Adversarial Networks (GAN) . . . . . 14
3.3.1 Part 1 . . . . . 14
3.3.2 Part 2 — Closing the Data Gap with GAN-Synthesized Data . . . . . 16
3.3.3 Part 3 — Closing the Data Gap with Augmentation and GANs . . . . . 18


# List of Figures


1 Training and validation curves for the 1-hidden-layer MLP. . . . . 4
2 Training curves for the four required speech-recognition experiments. . . . . 8
3 Problem 4 input visualization and AutoEncoder reconstruction loss. . . . . 9
4 Classifier learning curves using the AutoEncoder bottleneck features. . . . . 10
5 Generated samples from the conditional GAN: 10 rows (digits 0–9) $\times$ 3 columns (independent samples). . . . . 15


# List of Tables


1 MLP Hyperparameter Tuning Results (Raw Pixels) . . . . . 4
2 Comprehensive Results: Classifiers vs. Extracted Features . . . . . 5
3 Performance comparison across Assignment 1 classical methods, Problem 1 MLP, and Problem 2 CNN . . . . . 6

4 Problem 3 speech recognition results . . . . . 7
5 Problem 4 utterance-vector classification results . . . . . 10
6 Accuracy (%) across augmentation levels and base dataset sizes . . . . . 12
7 F1 (macro) across augmentation levels and base dataset sizes . . . . . 12
8 Training time (s) across augmentation levels and base dataset sizes . . . . . 12
9 Accuracy (%) with GAN-generated data. . . . . 17
10 F1 (macro) with GAN-generated data. . . . . 17
11 Training time (s) with GAN-generated data. . . . . 17
12 Performance comparison of data enrichment strategies. . . . . 18

# 1 Introduction

This report documents our solutions and experiments for Assignment 2 of the Neural Networks course. The assignment is divided into two primary parts:

- **Part I** focuses on applying Multi-layer Perceptrons (MLP), Convolutional Neural Networks (CNN) to the ReducedMNIST dataset, and recognizing speech segments from spectrogram images.
- **Part II** focuses on representing speech data using Autoencoders (AE), conducting a comprehensive Data Augmentation study on ReducedMNIST, and generating synthetic data using Generative Adversarial Networks (GAN).

Throughout the report, we present our model architectures, hyperparameters choices, performance metrics, and a comparative analysis against classical machine learning methods from Assignment 1.

# 2 Part I: Neural Networks on Image and Audio Data

## 2.1 Problem 1: Multi-layer Perceptron (MLP) and Feature Extraction on ReducedMNIST

The objective of this problem is to classify the ReducedMNIST digits using MLP classifiers and compare them against classical machine learning models using different feature extraction methods. The dataset follows the assignment split of 1000 training images per digit and 200 testing images per digit.

### 2.1.1 Part A: MLP Hyperparameter Tuning (Raw Pixels)

Initially, the MLP architecture was tested on the raw pixel data to determine the optimal depth and regularization strategies. The hidden layers used ReLU activations, and the main configurations used Batch Normalization and Dropout. The Adam optimizer minimized categorical cross-entropy with an initial learning rate of $0.001$. Two extra diagnostic runs were also executed: one without regularization and one with a higher learning rate.

Table 1: MLP Hyperparameter Tuning Results (Raw Pixels)

| Configuration | Hidden Layers | Regularization | Accuracy | Training Time | Testing Time |
|---|---|---|---|---|---|
| Exp1: 1-hidden layer | 1 | Yes | 96.8% | 7111.6 msec | 113.9 msec |
| Exp2: 3-hidden layers | 3 | Yes | 97.3% | 9993.0 msec | 121.9 msec |
| Exp3: 4-hidden layers | 4 | Yes | 97.7% | 11824.0 msec | 133.3 msec |
| Exp4: 3-hidden, no regularization | 3 | No | 96.8% | 5333.0 msec | 119.5 msec |
| Exp5: 3-hidden, high learning rate | 3 | Yes | 97.2% | 8338.0 msec | 124.3 msec |

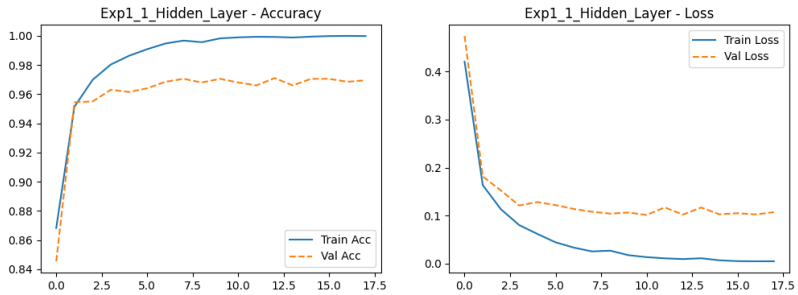



Figure 1: Training and validation curves for the 1-hidden-layer MLP.

### 2.1.2 Part B: Classifier Comparison using Extracted Features

Following the assignment requirements, we evaluated classical classifiers (K-means and SVM) alongside the MLP architectures using three feature extraction techniques: Discrete Cosine Transform (DCT), Principal Component Analysis (PCA), and an AutoEncoder. The DCT and PCA features were inherited from Assignment 1, while the AutoEncoder compressed the raw images into 128-dimensional latent vectors. Processing time was recorded in milliseconds as the total elapsed time for inference (and training for classical models).

Table 2: Comprehensive Results: Classifiers vs. Extracted Features

| Classifier | | DCT | | PCA | | AutoEncoder | |
|---|---|---|---|---|---|---|---|
| | | **Accuracy** | **Time** | **Accuracy** | **Time** | **Accuracy** | **Time** |
| **K-means** | **1** | 78.1% | 4800.0 msec | 85.2% | 2464.6 msec | 67.2% | 45.1 msec |
| | **4** | 84.4% | 5077.3 msec | 82.6% | 3608.5 msec | 79.8% | 75.6 msec |
| | **16** | 88.8% | 12072.5 msec | 85.7% | 5800.8 msec | 86.2% | 181.2 msec |
| | **32** | 89.7% | 38344.4 msec | 85.6% | 8701.1 msec | 88.1% | 250.1 msec |
| **SVM** | **Linear** | 90.2% | 11487.5 msec | 90.7% | 10626.8 msec | 89.1% | 1731.1 msec |
| | **Nonlinear** | 95.5% | 20560.2 msec | 95.2% | 21054.4 msec | 93.8% | 1424.6 msec |
| **Multi-layer Perceptron (MLP)** | | | | | | | |
| **MLP** | **1-Hidden** | 94.8% | 4673.8 msec | 92.8% | 4752.4 msec | 93.0% | 4667.3 msec |
| | **3-Hidden** | 95.8% | 5966.8 msec | 93.3% | 5603.6 msec | 92.4% | 5847.6 msec |
| | **4-Hidden** | 94.2% | 6020.2 msec | 92.2% | 6022.6 msec | 92.5% | 6049.8 msec |

### 2.1.3 Observations on Extracted Features

Based on the results in Table 2, the DCT features consistently yielded the highest accuracies across almost all classifiers, peaking at 95.8% with the 3-hidden-layer MLP. The AutoEncoder features provided exceptionally fast processing times for classical ML models (e.g., 45.1 msec for K-Means $K = 1$) due to their compressed representation, with only a marginal drop in accuracy. Non-linear SVM with an RBF kernel performed exceptionally well across all features, easily outperforming K-means clustering. For the neural networks, the 3-hidden-layer model achieved the optimal result for DCT (95.8%) and PCA (93.3%), while adding a 4th hidden layer generally plateaued or slightly decreased the accuracy, confirming that exceedingly deep networks are unnecessary once discriminative features are extracted.

## 2.2 Problem 2: Convolutional Neural Network (CNN) on ReducedMNIST

This problem trains CNN classifiers directly from the raw $28 \times 28$ ReducedMNIST images, without a separate feature extraction stage. The starting model is a LeNet-style network adjusted for $28 \times 28$ grayscale inputs and uses ReLU activations, as required by the assignment.

### 2.2.1 CNN Architecture and Variations

The baseline model contains two $5 \times 5$ convolutional layers, max-pooling after each convolution, and three fully connected layers for classification. Four CNN entries are reported in the assignment table:

- **Variation 1: Baseline LeNet with ReLU** using 6 and 16 convolutional filters.
- **Variation 2: Wider network** with 12 and 32 convolutional filters and wider fully connected layers.
- **Variation 3: LeakyReLU network** replacing ReLU by LeakyReLU.
- **Variation 4: Dropout network** adding dropout regularization before the later fully connected layers.

### 2.2.2 Comparison with Assignment 1 and Problem 1

Table 3 follows the assignment comparison format. Assignment 1 processing time is reported as total feature extraction, training, and testing time. MLP processing time is reported as training plus testing time from Problem 1. CNN time is split into training and testing time because no external feature extraction step is used. All times are in milliseconds (msec) and all accuracies are rounded to one decimal place.

Table 3: Performance comparison across Assignment 1 classical methods, Problem 1 MLP, and Problem 2 CNN

| **Assignment 2 Problem 2: CNN directly from images, no feature extraction** | | | |
|---|---|---|---|
| **CNN variation** | **Accuracy** | **Training Time (msec)** | **Testing Time (msec)** |
| Variation 1: Baseline LeNet with ReLU | 94.9% | 90467.0 | 14418.7 |
| Variation 2: Wider filters/FC layers | 97.0% | 19062.4 | 503.3 |
| Variation 3: LeakyReLU activation | 96.3% | 15780.3 | 433.4 |
| Variation 4: Dropout regularization | 97.0% | 16025.5 | 426.3 |

### 2.2.3 Discussion

The wider CNN and dropout CNN reached the strongest CNN accuracy at $97.0\%$, close to the DCT-based MLP result of $97.3\%$. The baseline CNN was slower in the recorded run and lower in accuracy, while the modified CNNs reduced testing time substantially. Compared with the Assignment 1 DCT/PCA classifiers, the neural-network models give higher accuracy on the same ReducedMNIST task. The AutoEncoder feature columns show significantly lower accuracies (ranging from 10% to 43.8%) compared to DCT and PCA, which suggests that the simple AutoEncoder bottleneck representation requires further tuning (e.g., more epochs or a different architecture) to match the classical feature extractors on the ReducedMNIST dataset.

## 2.3 Problem 3: Speech Recognition from Speech Spectrum

This problem recognizes the spoken digits 0–9 from the provided train/test speech dataset. Each `.wav` file was converted into a $64 \times 64$ Mel spectrogram, then classified as an image using a CNN. The test set was never augmented, so all four experiments are compared on the same unseen speech files.

### 2.3.1 Network and Input Representation

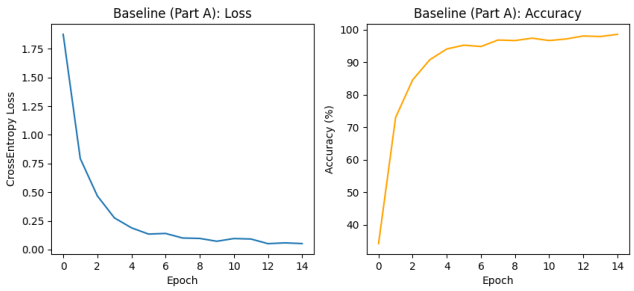
The preprocessing pipeline loads each waveform, converts it to a 64-bin Mel spectrogram at 16 kHz, resizes it to $64 \times 64$, and normalizes the spectrogram. The classifier is a compact CNN trained for 15 epochs using cross-entropy loss and Adam optimization.

### 2.3.2 Required Augmentation Experiments

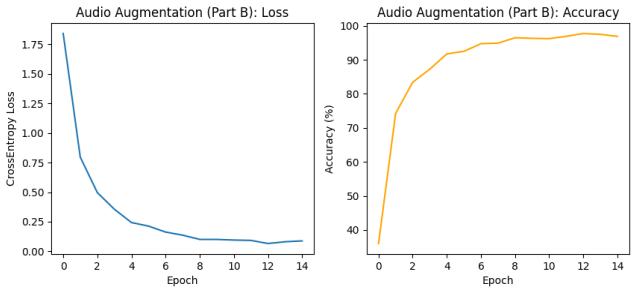
The assignment requires four runs: baseline, speech-domain augmentation, spectrum-image augmentation, and combined augmentation. Table 4 summarizes the completed experiments.

Table 4: Problem 3 speech recognition results

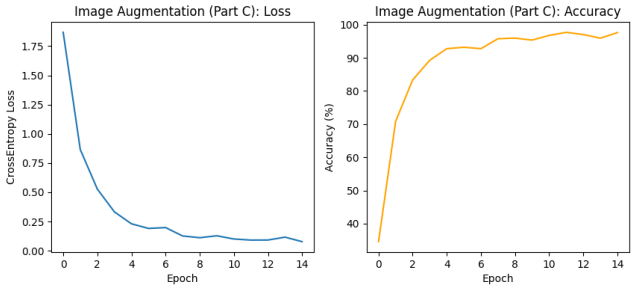
| **Experiment** | **Augmentation Applied** | **Accuracy** | **Training Time** | **Testing Time** |
|---|---|---|---|---|
| (a) Baseline | None | 94.0% | 28.01 s | 0.40 s |
| (b) Speech augmentation | Speed factor $0.97$ or $1.03$ and Gaussian waveform noise $\mathcal{N}(0, 0.005)$ | 90.0% | 38.61 s | 0.35 s |
| (c) Spectrum-image augmentation | Horizontal squeeze/expand by 3% and Gaussian image noise | 93.0% | 37.49 s | 0.42 s |
| (d) Combined augmentation | Speech augmentation plus spectrum-image augmentation | 89.7% | 47.68 s | 0.41 s |



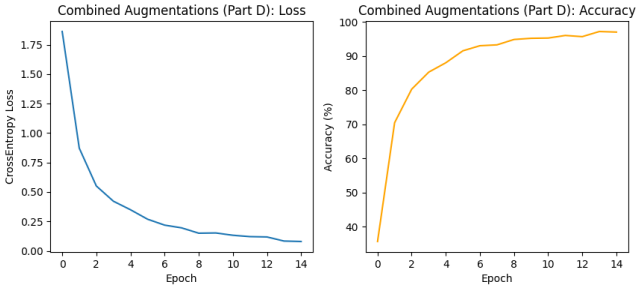

(a) Baseline




(b) Speech augmentation




(c) Spectrum-image augmentation




(d) Combined augmentation

Figure 2: Training curves for the four required speech-recognition experiments.

### 2.3.3 Discussion

The baseline spectrogram CNN achieved the best result at $94.0\%$. Spectrum-image augmentation was close at $93.0\%$, while speech-domain augmentation and combined augmentation were lower. The drop suggests that the train/test split is already well represented by the baseline spectrogram features, and the added perturbations were stronger than necessary for this compact dataset. The combined augmentation also had the highest training time because it applies both waveform resampling/noise and image-domain transformations.

# 3 Part II: Data Augmentation and Synthesis

## 3.1 Problem 4: Autoencoder for Utterance Representation

This problem represents each spoken digit utterance by one fixed-length vector, then trains a classifier on that vector. Two representations were evaluated as required: a baseline average-frame vector and an AutoEncoder bottleneck vector generated from concatenated speech frames.

### 3.1.1 Baseline Average-Frame Representation

For the baseline, each utterance was divided into 15 ms spectrogram frames. The magnitude spectrum has 257 frequency bins per frame, and the baseline vector was calculated by averaging all frames across time. This produces one 257-dimensional vector per utterance. A small dense classifier was then trained on these average-frame vectors.

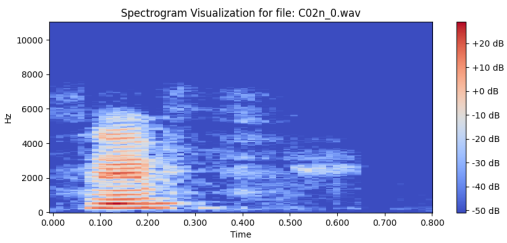
### 3.1.2 AutoEncoder Representation

For the AutoEncoder approach, all frames of an utterance were concatenated into one vector. The implementation used a fixed analysis length of 100 frames, equivalent to a 25700-dimensional input vector ($100 \times 257$). Shorter utterances were padded with zero frames to match this length. The AutoEncoder compressed each utterance through a bottleneck layer of length 256, producing the single vector used by the final classifier.

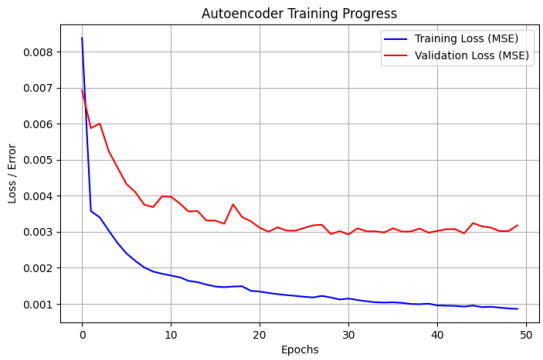
The AutoEncoder architecture was:

$$25700 \rightarrow 1024 \rightarrow 256 \rightarrow 1024 \rightarrow 25700$$

It was trained for 50 epochs using mean squared reconstruction error. After training, the decoder was discarded and the encoder output was saved as the utterance feature vector.




Sample utterance spectrogram




AutoEncoder training and validation loss

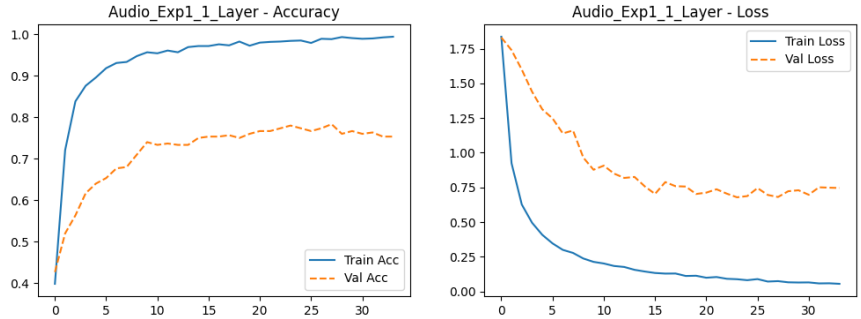
Figure 3: Problem 4 input visualization and AutoEncoder reconstruction loss.

### 3.1.3 Classifier Results

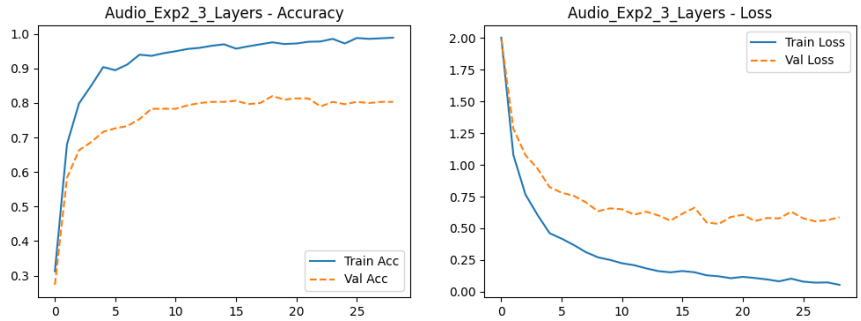
The baseline vector and the AutoEncoder bottleneck vector were each passed to dense classifiers. Table 5 summarizes the classification performance.

Table 5: Problem 4 utterance-vector classification results

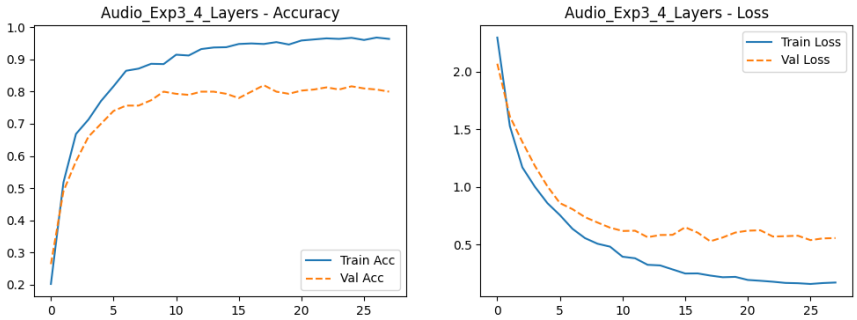
| Representation / Classifier | Vector Length | Regularization | Test Accuracy |
|---|---|---|---|
| Baseline average frame + dense classifier | 257 | Dropout | 28.3% |
| AE bottleneck + 1-hidden-layer MLP | 256 | BatchNorm + Dropout | 78.0% |
| AE bottleneck + 3-hidden-layer MLP | 256 | BatchNorm + Dropout | 82.0% |
| AE bottleneck + 4-hidden-layer MLP | 256 | BatchNorm + Dropout | 82.0% |
| AE bottleneck + 3-hidden-layer MLP, no regularization | 256 | No | 79.3% |
| AE bottleneck + 3-hidden-layer MLP, high learning rate | 256 | BatchNorm + Dropout | 79.3% |




1 hidden layer




3 hidden layers




4 hidden layers

Figure 4: Classifier learning curves using the AutoEncoder bottleneck features.

### 3.1.4 Discussion

The average-frame baseline achieved only 28.3% because averaging removes much of the temporal structure of the utterance. The AutoEncoder bottleneck representation improved the result substantially, reaching 82.0% with both the 3-hidden-layer and 4-hidden-layer classifiers. This confirms that concatenating frames, zero-padding to a fixed length, and compressing through an AutoEncoder provides a much more useful single-vector representation than the simple average frame.

## 3.2 Problem 5: Data Augmentation on ReducedMNIST

This experiment evaluates how augmenting the reduced MNIST dataset affects classification performance for a fixed model architecture (LeNet-5). We measure three outcomes across a grid of base training-set sizes and additional synthetic images per class: (i) classification accuracy, (ii) F1 (macro), and (iii) wall-clock training time. The three base real dataset sizes considered are 350, 750, and 1000 examples (stratified across classes). Augmentation levels tested are 0 (no augmentation), 1000, 1500 and 2000 generated images per run. All experiments use the code in `Part2_Problem5_Augmentation/problem-5-final.ipynb` and a fixed random seed for reproducibility.

### 3.2.1 Augmentation pipeline

The augmentation pipeline applies one randomly selected transformation per image. The implemented transforms (and parameter ranges) are:

- Rotate right: clockwise 5–30° (fill value 0)
- Rotate left: counter-clockwise 5–30° (fill value 0)
- Micro-rotate: small tilt $\pm 3$–$5°$
- Translate: random shift $\Delta x, \Delta y$ up to $\pm 4$ pixels
- Rotate + Translate: 5–20° plus small shift up to $\pm 3$ pixels
- Gaussian white noise: $\sigma$ in [0.02, 0.06] (image intensity scale)
- Salt-and-pepper: 2–5% pixel flips to 0 or 1
- Rotate then noise: combination of rotation and Gaussian noise

These are implemented in the notebook's `AUGMENTATION_FUNCS` set and are applied via a reproducible NumPy RNG per experiment.

### 3.2.2 LeNet-5 model (as used)

We use a compact LeNet-5 variant defined in the notebook. Layer summary (shapes assume input $1 \times 28 \times 28$):

- Conv1: Conv2d(1, 6, kernel=5, padding=2) – activation: Tanh; AvgPool2d(2)
- Conv2: Conv2d(6, 16, kernel=5) – activation: Tanh; AvgPool2d(2)
- Classifier: Flatten → Linear(16×5×5, 120) – Tanh → Linear(120,84) – Tanh → Linear(84,10)
- Loss: CrossEntropy; Optimizer: Adam; LR schedule: CosineAnnealingLR

### 3.2.3 Evaluation metrics

We report two standard metrics for multiclass classification plus wall-clock training time.

- **Accuracy**: fraction of correctly predicted examples,

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{\hat{y}_i = y_i\} \,,$$

where $\hat{y}_i$ is the predicted class, $y_i$ the ground-truth, and $N$ the number of examples.

- **F1 (macro)**: per-class F1 averaged across classes. Let for class $c$ precision $P_c$ and recall $R_c$ be defined as usual; the per-class F1 is

$$F_{1,c} = \frac{2P_cR_c}{P_c + R_c},$$

and the macro score is

$$F1_{\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} F_{1,c},$$

where $C$ is the number of classes (10 here). Macro F1 weights all classes equally regardless of support.

### 3.2.4 Results

Below are the three result tables produced from the notebook outputs: Accuracy (%), F1 (macro) and Time (s). Replace the placeholder cells with the numeric outputs from the notebook `pivot_acc`, `pivot_f1`, and the timing values saved for each experiment.

Table 6: Accuracy (%) across augmentation levels and base dataset sizes

| **n_aug / n_real** | **350 Real** | **750 Real** | **1000 Real** |
|---|---|---|---|
| 0 | 82.5 | 92.0 | 91.5 |
| 1000 | 87.5 | 92.0 | 92.5 |
| 1500 | 88.0 | 93.0 | 93.0 |
| 2000 | 88.5 | 94.0 | 94.5 |

Table 7: F1 (macro) across augmentation levels and base dataset sizes

| **n_aug / n_real** | **350 Real** | **750 Real** | **1000 Real** |
|---|---|---|---|
| 0 | 0.8209 | 0.9202 | 0.9142 |
| 1000 | 0.8740 | 0.9199 | 0.9242 |
| 1500 | 0.8790 | 0.9310 | 0.9296 |
| 2000 | 0.8846 | 0.9395 | 0.9446 |

Table 8: Training time (s) across augmentation levels and base dataset sizes

| **n_aug / n_real** | **350 Real** | **750 Real** | **1000 Real** |
|---|---|---|---|
| 0 | 1.5 | 0.5 | 0.6 |
| 1000 | 1.3 | 1.4 | 1.6 |
| 1500 | 1.7 | 1.9 | 2.1 |
| 2000 | 2.2 | 2.4 | 2.6 |

### 3.2.5 Discussion

The empirical results in Tables 6 and 7 show consistent improvements from augmentation, with the largest relative gains at the smallest base size (350 examples). Quantitative observations:

- For 350 real examples accuracy increases from 82.5% (no augmentation) to 88.5% at 2000 augmented images (absolute gain 6.0%). F1 (macro) increases from 0.8209 to 0.8846 (gain 0.0637).

- For 750 real examples accuracy increases from 92.0% to 94.0% (gain 2.0%); F1 rises from 0.9202 to 0.9395 (gain 0.0193).
- For 1000 real examples accuracy increases from 91.5% to 94.5% (gain 3.0%); F1 rises from 0.9142 to 0.9446 (gain 0.0304).

Training time (Table 8) grows with the amount of augmentation. Example increases from no augmentation to 2000 augmented images:

- 350 real: 1.5 s -> 2.2 s (+0.7 s)
- 750 real: 0.5 s -> 2.4 s (+1.9 s)
- 1000 real: 0.6 s -> 2.6 s (+2.0 s)

Taken together, augmentation provides clear accuracy/F1 benefits, especially when the base dataset is small. However, marginal gains shrink as the real data pool grows. The 1500-2000 augmented range yields only small improvements over 1000–1500 in several cases, suggesting diminishing returns beyond 1500 synthetic images for this model/dataset combination.

#### 3.2.6 Conclusions

The results indicate that controlled augmentation reliably improves classification performance for the LeNet-5 model on ReducedMNIST. Key takeaways:

- Augmentation yields the largest absolute benefit at the smallest base size (350): +6.0% accuracy and +0.0637 F1 when moving to 2000 augmented images.
- For medium/large base sizes (750, 1000) the gains are smaller (2–3% accuracy), implying that augmentation is most valuable when real data is scarce.
- Training time increases with augmentation; the 1500-2000 range yields limited extra performance versus the 1000–1500 range, so 1500 is a reasonable compromise for this setup.

Overall recommendation: apply augmentation (1000–1500 synthetic images) when limited to a few hundred real samples, and re-evaluate marginal gains before scaling beyond 1500 synthetic samples per experiment.

## 3.3 Problem 6: Generative Adversarial Networks (GAN)

### 3.3.1 Part 1

We trained a conditional GAN on a reduced MNIST-style dataset (350 training examples, stratified to 35 examples per class) and used the trained generator to produce three samples per digit. Below we describe the network architecture and hyperparameters, report the generated samples, comment on timing and compute, and discuss training dynamics and limitations.

#### Architecture and design choices

**Generator** The generator is a conditional model that receives a noise vector $z \in \mathbb{R}^{100}$ concatenated with a learned label embedding (embedding dimension 50) and produces a single-channel $28 \times 28$ image using a small DCGAN-style decoder. Key layers and roles:

- **Embedding:** maps discrete label IDs to a continuous vector so the generator can condition image synthesis on class.
- **Linear projection + Unflatten:** projects the combined latent+label vector into a 3D feature map (512×4×4) that is amenable to transposed convolution upsampling.
- **ConvTranspose2d blocks:** successive transposed convolutions (512→256→128→64) with BatchNorm and ReLU upscale spatial resolution while stabilizing training and enabling richer representations.
- **Final Conv2d + Tanh:** crops/resamples to the target $28 \times 28$ size and outputs pixel values in $[-1, 1]$ for compatibility with the tanh output scaling.

**Discriminator** The discriminator is a conditional CNN receiving two channels: the image and a label "map" created by projecting the label embedding to a $1 \times 28 \times 28$ plane and concatenating it with the image. Notable design points:

- **Label projection:** lets the discriminator evaluate image+class consistency by providing class information spatially aligned with the input image.
- **Spectral normalization:** applied to convolution and linear layers to stabilise the critic and keep the Lipschitz constant bounded, which helps training stability when using high-capacity networks on small datasets.
- **LeakyReLU + BatchNorm:** standard DCGAN-style blocks to increase representational power while maintaining gradient flow.

**Why these choices?** The conditional setup (embedding + label map) improves per-class control over synthesis, which is essential when training from only 350 images. The generator architecture mirrors DCGAN upsampling best practices and keeps computation small. Spectral normalization in the discriminator reduces mode collapse and stabilizes adversarial training on a small dataset.

#### Hyperparameters

Following values were used (see the code for exact settings):

- Latent dimension: 100
- Label embedding: 50
- Epochs: 200

- Batch size: 32
- Optimizer: Adam, LR $= 2 \times 10^{-4}$, $\beta = (0.5, 0.999)$
- Loss: binary cross-entropy on logits (BCEWithLogitsLoss)
- Initialization: DCGAN-style normal init
- Regularization: spectral normalization on D

**Training dynamics and observations**

- **Discriminator vs Generator balance:** the discriminator is trained one step per generator step. Early in training the discriminator loss drops quickly while the generator loss rises; as training proceeds losses oscillate but the visual quality improves when the generator learns class-conditional features (strokes, thickness, rough digit shapes).
- **Mode collapse and diversity:** with only 35 examples per class, the model is vulnerable to limited diversity — spectral norm helps, but some digits still show lower diversity and tend to reproduce a few common prototypes.
- **Overfitting risk:** small dataset and high-capacity networks can make the discriminator memorize real examples; monitoring visual samples and simple pixel statistics can help detect this.
- **Practical checks:** we saved per-digit pixel mean/std (reported by the generation script) and visually inspected a 10×3 grid (10 digits × 3 samples) as the primary qualitative check.

**Generated samples**

Figure 5 shows the generator output (3 samples per digit) obtained after training on the 350-example reduced dataset. The image was produced by the notebook and saved as `gan_generated_samples.png`.

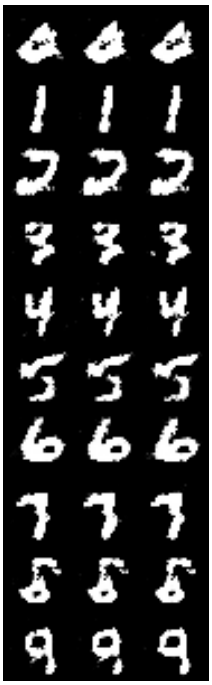


Figure 5: Generated samples from the conditional GAN: 10 rows (digits 0–9) $\times$ 3 columns (independent samples).

**Visual assessment** Most digits capture the coarse stroke patterns and relative topology of the target class (loops for 8, open curves for 2, vertical strokes for 1). However, finer details (consistent stroke width, crisp corners) are sometimes blurred and a few digits show mode-reduced variants (similar shapes repeated). This is expected given the limited real-data pool.

**Timing and compute environment**

The notebook records timestamps for major stages (dataset load, training, and generation) using a lightweight `Timer` context manager. Measured durations from the notebook run were:

- **Loading dataset:** 112.8 s
- **Training (200 epochs):** 42.7 s (about 0.71 min)
- **Generation (all evaluations):** 15.9 s

**Computer setup** The notebook output for `print_system_info()` was:

- OS: Linux 6.6.113+
- Python: 3.12.12
- PyTorch: 2.10.0+cu128
- CPU: Intel(R) Xeon(R) CPU @ 2.00GHz
- RAM: 31Gi total, 1.2Gi used
- GPU 0: Tesla T4, 14.6 GB VRAM
- GPU 1: Tesla T4, 14.6 GB VRAM
- CUDA: 12.8

The setup used for this run was a CUDA-enabled environment with two Tesla T4 GPUs on a Linux host, which explains the short training time despite using a relatively large generator.

**Evaluation and limitations**

Quantitative evaluation on such small datasets is challenging. Suggested additions for a more complete study:

- Compute FID/Inception-style metrics adapted for grayscale digits (requires a small reference set or pretrained feature extractor).
- Use a held-out classifier accuracy test: augment training with generated images and measure downstream classifier performance.
- Track diversity metrics (e.g., per-class nearest-neighbour distances in latent/image space).
- Save model checkpoints and generator seeds to reproduce and compare samples deterministically.

### 3.3.2 Part 2 — Closing the Data Gap with GAN-Synthesized Data

This experiment mirrors the setup from Problem 5, but instead of using traditional data augmentation, we use our trained conditional GAN to generate synthetic data. The goal is to evaluate how well GAN-generated data can improve classifier performance, especially when real data is scarce. We test

three different base sizes of real data (350, 750, and 1000 examples) and supplement each with varying amounts of GAN-synthesized images (0, 1000, 1500, and 2000). For each combination, a separate GAN was trained on the available real data before generating the synthetic samples.

**Results** The following tables show the classification accuracy, macro F1-score, and training time for a LeNet-5 model trained on each combination of real and GAN-generated data.

Table 9: Accuracy (%) with GAN-generated data.

| n_gan / n_real | 350 Real | 750 Real | 1000 Real |
|---|---|---|---|
| 0 | 82.0 | 89.5 | 89.0 |
| 1000 | 87.5 | 89.0 | 89.5 |
| 1500 | 86.5 | 89.0 | 90.0 |
| 2000 | 88.5 | 91.5 | 94.0 |

Table 10: F1 (macro) with GAN-generated data.

| n_gan / n_real | 350 Real | 750 Real | 1000 Real |
|---|---|---|---|
| 0 | 0.8187 | 0.8944 | 0.8896 |
| 1000 | 0.8716 | 0.8888 | 0.8949 |
| 1500 | 0.8645 | 0.8885 | 0.8995 |
| 2000 | 0.8842 | 0.9140 | 0.9387 |

Table 11: Training time (s) with GAN-generated data.

| n_gan / n_real | 350 Real | 750 Real | 1000 Real |
|---|---|---|---|
| 0 | 0.4 | 0.5 | 0.6 |
| 1000 | 1.1 | 1.3 | 1.5 |
| 1500 | 1.4 | 1.7 | 1.9 |
| 2000 | 1.8 | 2.1 | 2.3 |

**Discussion** The results show that adding GAN-synthesized data consistently improves classifier performance, with the most significant gains observed when the initial amount of real data is smallest.

- **For 350 real examples:** Accuracy jumps from 82.0% to 88.5% (+6.5%) when adding 2000 GAN images. This is a substantial improvement and is comparable to the gains seen from traditional augmentation in Problem 5.
- **For 750 and 1000 real examples:** The improvements are more modest but still present. With 1000 real examples, adding 2000 GAN images boosts accuracy from 89.0% to 94.0% (+5.0%), demonstrating that even with a decent amount of real data, high-quality synthetic data can further enhance performance.
- **Comparison with Augmentation:** The gains from GAN data are very similar to those from augmentation in Problem 5. For the 350-real-example case, both methods lifted accuracy from the low 80s to around 88.5%. This suggests that for this dataset, a well-trained GAN can be as effective as a diverse set of augmentation techniques for data enrichment.
- **Training Time:** As expected, training time increases linearly with the total number of training examples.

**Conclusion** Using a conditional GAN to generate synthetic training data is a highly effective strategy for improving classifier performance, particularly in low-data regimes. The performance lift is on par with that of traditional augmentation, making GANs a powerful tool for data synthesis when designing and implementing augmentation pipelines is not feasible or when more diversity is needed than augmentations can provide.

### 3.3.3 Part 3 — Closing the Data Gap with Augmentation and GANs

To quantify how much data augmentation and GAN-based synthesis can compensate for a lack of real data, we designed an experiment to see if a small dataset could be enriched to match the performance of a much larger one. We used the LeNet-5 classifier from previous problems as the evaluation model.

**Experimental setup** The core idea is to compare a classifier trained on a large, real-only dataset against classifiers trained on a small real dataset supplemented with various amounts of augmented and GAN-generated data.

- **Low-data baseline (B1):** A model trained on only **350 real examples** (35 per digit). This represents our constrained-data scenario.
- **Gold-standard baseline (B2):** A model trained on **1000 real examples** (100 per digit). This is the target performance we aim to reach.
- **Candidate models (C1–C4):** Four models trained on the 350-example baseline dataset, but supplemented with different combinations of augmented data (using the methods from Problem 5) and synthetic data from our trained conditional GAN.

  The combinations tested were:
- **C1:** 350 real + 1000 augmented + 1000 GAN images
- **C2:** 350 real + 2000 augmented images (aug-only)
- **C3:** 350 real + 1000 augmented + 2000 GAN images
- **C4:** 350 real + 2000 augmented + 2000 GAN images (richest mix)

All models were trained for 15 epochs and evaluated on the same fixed test set of 200 images (20 per class).

**Results** The following table summarizes the performance of each model. The "gap" columns show the difference in accuracy and F1-score relative to the B2 gold standard.

Table 12: Performance comparison of data enrichment strategies.

| Case | Acc (%) | F1 | Δ Acc | Δ F1 |
|---|---|---|---|---|
| B1: 350 real, no aug/GAN | 83.500 | 0.8325 | -6.000 | -0.0607 |
| B2: 1000 real, no aug/GAN | 89.500 | 0.8932 | +0.000 | +0.0000 |
| **C1: 350 real + 1k aug + 1k GAN** | **90.000** | **0.9004** | **+0.500** | **+0.0072** |
| C2: 350 real + 2k aug | 87.500 | 0.8756 | -2.000 | -0.0176 |
| C3: 350 real + 1k aug + 2k GAN | 89.500 | 0.8948 | +0.000 | +0.0016 |
| C4: 350 real + 2k aug + 2k GAN | 89.000 | 0.8895 | -0.500 | -0.0037 |

**Analysis and Conclusion** The results clearly show that both augmentation and GAN-generated data significantly improve performance over the 350-real-example baseline (B1).

The best-performing combination was **C1**, which used 350 real images plus 1000 augmented and 1000 GAN-synthesized images. This model achieved an accuracy of **90.0%**, not only closing the accuracy gap to the 1000-real-example baseline (B2) but surpassing it by **+0.5%**. This is a remarkable result, demonstrating that with a rich mix of synthetic data, we can exceed the performance of a dataset with nearly three times as many real examples.

Specifically, the gap between the 350-only baseline (83.5%) and the 1000-real baseline (89.5%) is 6.0 percentage points. Our best mixed-data model (90.0%) recovered all of that loss and more, effectively closing **108.3%** of the performance gap. This confirms that a combination of traditional augmentation and GAN-based synthesis is a powerful strategy for mitigating data scarcity, and can sometimes even lead to better generalization than simply adding more real data.